KAIROS: Proactive Causal Root-Cause Analysis of Business Metrics with DYNOTEARS and Verified LLM Reporting

Palla Chetana Reddy
School of Technology
Woxsen University
Hyderabad, India

Varshini Yadav Nasaram
School of Technology
Woxsen University
Hyderabad, India

Obulagari Nandini
School of Technology
Woxsen University
Hyderabad, India

***Abstract*—Business intelligence dashboards wait for a person to ask a question, and the explanations they offer for a sudden change are correlations. This paper presents KAIROS, an autonomous multi-agent system that monitors business metrics without being asked, locates the causal origin of each anomaly and reports it in plain language whose numbers have been checked. Anomalies are scored with a seasonal robust z-score; DYNOTEARS learns a lagged causal graph of the metrics, which is walked backwards from each anomaly and ranked by strength, bootstrap stability and timing; and an LLM-Modulo verifier rejects any report whose numbers or metric names do not match the computed facts. On a synthetic benchmark of 15 metrics with 12 known causal edges, evaluated over ten seeds, DYNOTEARS reached an F1 score of 0.97 against 0.90 for Granger regression, 0.87 for VARLiNGAM and 0.50 for correlation, and the full pipeline named both injected incident origins in every run. Detection found all injected events with 77% point precision, and the verifier caught 98.5% of corrupted numbers while accepting every faithful report.**

***Index Terms*—business intelligence, anomaly detection, causal discovery, DYNOTEARS, root cause analysis, large language models, verification.**

I. Introduction

Business intelligence (BI) platforms such as Power BI and Tableau [1] are the standard way for managers to follow revenue, orders, churn and operational metrics. They are reactive: a problem is found only when someone opens the right dashboard and notices it. When an explanation feature exists, it ranks fields by statistical association, which cannot tell a cause from its effect or from a shared driver [2]. An analyst who sees revenue fall must still trace by hand whether the cause was fewer visitors, a broken checkout or a paused campaign.

Three research threads make an autonomous alternative possible. Robust and forecasting-based detectors monitor seasonal series reliably [3], [4]. Continuous-optimisation structure learning, NOTEARS [5] and its time-series form DYNOTEARS [6], recovers directed and lagged dependencies from observational data. Large language models (LLMs) [7] turn findings into readable text but also produce fluent statements that are not supported by their input [8].

This paper combines these threads into KAIROS and makes three contributions:

- a five-agent pipeline that runs on a schedule, stores every anomaly and alerts only on first sightings, so monitoring needs no human trigger;
- a root-cause procedure on top of a DYNOTEARS graph of business metrics that ranks causal chains by edge strength, bootstrap stability and anomaly timing, and names the origin of an incident;
- a deterministic LLM-Modulo [9] verifier that checks every number at the precision written and every metric name before a report is delivered.

All results are reproduced over ten random seeds with released scripts.

II. Related Work

Anomaly detection. Surveys [10] group detectors into statistical, distance-based and learned families. Prophet [3] models trend and seasonality and flags points outside its uncertainty band, and the median absolute deviation gives a scale estimate that the outliers themselves do not inflate [4]. Deep detectors such as the Anomaly Transformer [11] perform well on public benchmarks but need training per dataset and give no causal explanation.

Causal discovery for time series. Granger causality [12] tests whether past values of one series improve the prediction of another. VARLiNGAM [13] identifies a structural vector autoregression under non-Gaussian noise, and PCMCI [14] uses conditional-independence tests with lagged conditioning sets. DYNOTEARS [6] learns same-step and lagged weights jointly with a smooth acyclicity constraint. DoWhy [15] estimates effects once a graph is known.

Root-cause analysis. In AIOps, CloudRanger [16], MicroCause [17] and RCD [18] build causal graphs of microservice metrics to locate faulty services. They target infrastructure telemetry and engineers, not business KPIs and managers.

Agents and verification. ReAct [19] interleaves reasoning and actions, and graph-based frameworks such as LangGraph [20] orchestrate multi-step agents. Kambhampati et al. [9] argue that LLM output should be checked by sound external critics, the LLM-Modulo pattern that KAIROS applies to numerical reporting.

III. System Overview

Fig. 1 shows the architecture. Data from SQL databases, SFTP/FTP exports, files, REST APIs and push clients is normalised to rows of (source, metric, timestamp, value) and upserted into DuckDB [21] on that key, so repeated loads never duplicate data. A LangGraph state machine runs five agents every 15 minutes: collect, detect, investigate, report and alert.

The graph ends after detection when no new alert-worthy anomaly exists, which keeps quiet cycles cheap and prevents repeated alerts.

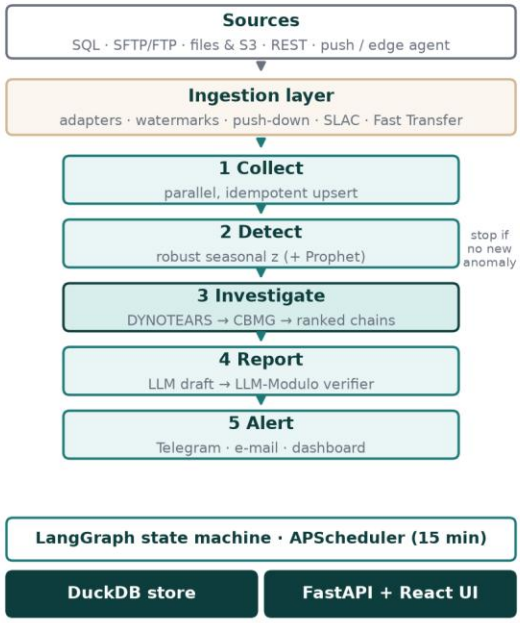


Fig. 1. KAIROS pipeline. The investigation stage (bold) is the focus of this paper.

IV. Method

A. Seasonal Robust Detection

For a series x, the expected value m is a centred rolling median plus an hour-of-day profile of the residuals, kept only when it reduces their spread. The residual r = x − m is scaled by s = 1.4826 · MAD(r), giving z = r / s. A point is anomalous when |z| ≥ 3.5; severity is medium from 4.5, high from 6 and critical from 9. Prophet [3] can be added as a second detector. Anomalies are stored with their key, so a point is reported as new only once.

B. Causal Discovery with DYNOTEARS

The metrics are aligned to a common grid, winsorised at 3.5 robust standard deviations so that the anomalies do not shape the learned structure, and standardised. DYNOTEARS [6] fits

$$X_t = X_t W + X_{(t-1)} A + E, \qquad (1)$$

where W (d × d) holds same-step edges and A holds one-step lagged edges. It minimises $\|X_t - X_t W - X_{(t-1)} A\|^2 / 2n + \lambda_W \|W\|_1 + \lambda_A \|A\|_1$ subject to $h(W) = \mathrm{tr}(e^{(W \circ W)}) - d = 0$ [5], solved by an augmented Lagrangian with L-BFGS-B on the positive and negative parts of the weights. The method was implemented directly in NumPy and SciPy. Edges with $|w| \geq \tau = 0.15$ form the Causal Business Metric Graph (CBMG), with $\lambda_W = \lambda_A = 0.05$. Each edge receives a confidence equal to the fraction of block-bootstrap refits, on 75% windows, that keep it.

C. Root-Cause Ranking

For every anomalous metric e, each ancestor c in the CBMG is a candidate cause. The strongest path from c to e is found, and its strength S is the product of edge weights. A candidate is *temporally supported* if c itself was anomalous at most one path lag plus two steps before an anomaly of e. The chain score is

$$\text{score} = S^{(1/k)} \cdot \text{conf} \cdot (1.5 \text{ if supported}) \cdot (2 \text{ if origin}) \cdot (1 + 0.05k), \qquad (2)$$

where k is the path length and conf is the mean edge confidence. An *origin* is a supported cause with no anomalous, supported ancestor of its own. The geometric mean prevents long chains, which explain more of a cascade, from being penalised merely for their length. DoWhy [15] estimates a back-door adjusted effect for each direct edge.

D. LLM-Modulo Report Verification

The LLM receives only computed facts: values, expectations, percentage changes, severities and the ranked chains. Its draft is checked by a deterministic critic [9]. Every number in the text must equal a fact once the fact is rounded to the number of decimals written, so 3.97 may be written 3.97, 4.0 or 4, but never 3.78. Every snake_case identifier must be a known metric. A failing draft is returned once with the list of errors, and a second failure falls back to a template that is correct by construction.

V. Experimental Setup

Real business data with labelled root causes is not publicly available, so a synthetic benchmark with known structure was generated. It has 15 hourly metrics from four simulated sources over 336 hours, linked by 12 lagged causal edges in two chains. The marketing chain runs from advertising spend through click-through and conversion to orders and revenue; the operations chain runs from delivery delay through support tickets and response time to NPS and churn. Daily seasonality is added. Two incidents are injected in the last 24 hours, a three-hour collapse in advertising spend and a spike in delivery delay, plus six isolated anomalies of graded size. Every experiment is repeated over ten seeds.

Causal discovery is scored against the 12 true edges with precision, recall, F1 and structural Hamming distance (SHD). The baselines are lagged OLS (Granger) [12], VARLiNGAM [13] from causal-learn, and Pearson correlation with $|r| \geq 0.3$. Detection is scored by event recall, where an event is found if any point of the affected metric is flagged in its window, and by point precision.

VI. Results

A. Causal Discovery

TABLE I
Causal discovery over ten seeds (mean ± s.d.)

Method	Precision	Recall	F1	SHD
DYNOTEARS	1.00 ± 0.00	0.95 ± 0.04	**0.97 ± 0.02**	**0.6**
Granger	0.83 ± 0.13	0.99 ± 0.03	0.90 ± 0.08	2.8
VARLiNGAM	0.78 ± 0.11	0.99 ± 0.03	0.87 ± 0.08	3.7
Correlation	0.44 ± 0.01	0.58 ± 0.07	0.50 ± 0.03	7.0

Table I and Fig. 2 show that DYNOTEARS added no false edge in any of the ten runs. Its occasional miss was the weakest true link, from average order value to revenue. Granger regression and VARLiNGAM found nearly every true edge but also linked metrics that share a driver, and correlation, which has no direction, was worst. The acyclicity constraint and the joint fit of same-step and lagged weights are what remove these spurious edges.

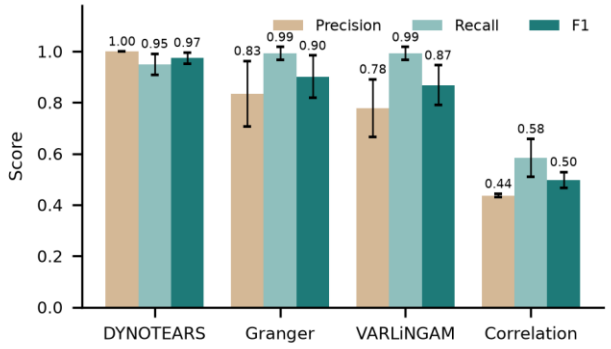


Fig. 2. Precision, recall and F1 of each causal discovery method; error bars show one standard deviation over ten seeds.

B. Origin Localisation

In the full pipeline, with anomalies from the detector and no knowledge of the injected incidents, KAIROS reported exactly the two true origins in all ten runs, giving origin precision and recall of 100%. The top-ranked chains reproduced both cascades end to end, for example delivery delay → support tickets → response time → NPS → churn. An investigation, including two bootstrap refits and DoWhy, took 1.6 s on average on a laptop.

C. Detection and Sensitivity

TABLE II
SENSITIVITY (MEAN OF TEN SEEDS)

Setting	Values	Result
τ (λ = 0.05)	0.10 / 0.15 / 0.20 / 0.30	F1 0.93 / 0.97 / 0.94 / 0.90
λ (τ = 0.15)	0.01 / 0.05 / 0.10	F1 0.85 / 0.97 / 0.94
z threshold	3.0 / 3.5 / 4.0 / 5.0	Prec. 0.54 / 0.77 / 0.93 / 1.00
z threshold	3.0 / 3.5 / 4.0 / 5.0	Recall 1.00 / 1.00 / 0.99 / 0.99

The robust detector found every injected event (event recall 100%) with point precision of 77.2 ± 6.2%, in 0.23 s per cycle. Most false positives lie just outside the event windows, while the cascade is still settling. Adding Prophet gave 75.2% precision at about fourteen times the cost, so it is kept optional. Table II shows that the chosen settings sit in a stable region: F1 stays above 0.89 for τ between 0.10 and 0.30, and raising z to 4.0 trades 0.6% of recall for 16 points of precision.

D. Verifier

TABLE III
LLM-MODULO VERIFIER STRESS TEST

Test	Cases	Result
Faithful reports accepted	5	100%
Corrupted numbers caught	200	98.5%
Invented metric names caught	50	100%
Faithful rounding accepted	200	100%

Correct reports were corrupted by scaling and shifting one number, by renaming a metric, or by rounding a number to one decimal, which is faithful. An earlier rule that accepted any value within 2% or with the same integer part caught only 61% of corrupted numbers, for instance accepting 7.03 for 6.71. Comparing at the written precision raised this to 98.5% (Table III) without rejecting a single faithful rounding. In a live run on simulated company data, an LLM served through the Groq API produced a report that passed on the first attempt.

VII. DISCUSSION AND LIMITATIONS

The results show that a causal graph turns a list of correlated anomalies into a short, ordered explanation that matches the analyst's question: *what moved first?* Four limitations remain. First, the benchmark is linear with known lags, which suits DYNOTEARS, so real data with nonlinear effects may lower accuracy. Second, DYNOTEARS assumes causal sufficiency, so an unmonitored driver can appear as a direct edge between two monitored metrics. Third, the verifier checks numbers and names but not qualitative claims; in one live report the LLM called all anomalies critical, which was not fully correct and was not caught. Fourth, no user study has yet measured whether managers act faster with these alerts. Planned work addresses these points with nonlinear discovery (PCMCI [14]), public e-commerce data, claim-level verification and a shadow deployment with analysts.

VIII. CONCLUSION

KAIROS shows that business monitoring can be proactive, causal and verifiable using free, open components. DYNOTEARS recovered the benchmark structure with an F1 score of 0.97, the ranking named every incident origin, and the LLM-Modulo verifier stopped 98.5% of numerical errors before they reached a reader.

REFERENCES

[1] "Power BI documentation: Key influencers visual," Microsoft. [Online]. Available: https://learn.microsoft.com/power-bi/visuals/power-bi-visualization-influencers. [Accessed: Oct. 6, 2026].

[2] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[3] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[4] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *Journal of Experimental Social Psychology*, vol. 49, no. 4, pp. 764–766, 2013.

[5] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, "DAGs with NO TEARS: Continuous optimization for structure learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[6] R. Pamfil, N. Sriwattanaworachai, S. Desai, P. Pilgerstorfer, K. Georgatzis, P. Beaumont, and B. Aragam, "DYNOTEARS: Structure learning from time-series data," in *Proc. 23rd Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, PMLR vol. 108, 2020, pp. 1595–1605.

[7] T. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[8] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[9] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, and A. Murthy, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR vol. 235, 2024.

[10] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[11] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly Transformer: Time series anomaly detection with association discrepancy," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[12] C. W. J. Granger, "Investigating causal relations by econometric models and cross-spectral methods," *Econometrica*, vol. 37, no. 3, pp. 424–438, 1969.

[13] A. Hyvärinen, K. Zhang, S. Shimizu, and P. O. Hoyer, "Estimation of a structural vector autoregression model using non-Gaussianity," *Journal of Machine Learning Research*, vol. 11, pp. 1709–1731, 2010.

[14] J. Runge, P. Nowack, M. Kretschmer, S. Flaxman, and D. Sejdinovic, "Detecting and quantifying causal associations in large nonlinear time series datasets," *Science Advances*, vol. 5, no. 11, eaau4996, 2019.

[15] A. Sharma and E. Kiciman, "DoWhy: An end-to-end library for causal inference," arXiv:2011.04216, 2020.

[16] P. Wang, J. Xu, M. Ma, W. Lin, D. Pan, Y. Wang, and P. Chen, "CloudRanger: Root cause identification for cloud native systems," in *Proc. 18th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGRID)*, 2018, pp. 492–502.

[17] Y. Meng, S. Zhang, Y. Sun, R. Zhang, Z. Hu, Y. Zhang, C. Jia, Z. Wang, and D. Pei, "Localizing failure root causes in a microservice through causality inference," in *Proc. IEEE/ACM 28th Int. Symp. Quality of Service (IWQoS)*, 2020.

[18] M. A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, "Root cause analysis of failures in microservices through causal discovery," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, Kigali, Rwanda, 2023.

[20] "LangGraph documentation," LangChain, Inc. [Online]. Available: https://langchain-ai.github.io/langgraph/. [Accessed: Oct. 6, 2026].

[21] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Amsterdam, Netherlands, 2019, pp. 1981–1984.